

Lecture 2

Creational design patterns Factory Method & Abstract Factory

Learning Objectives:

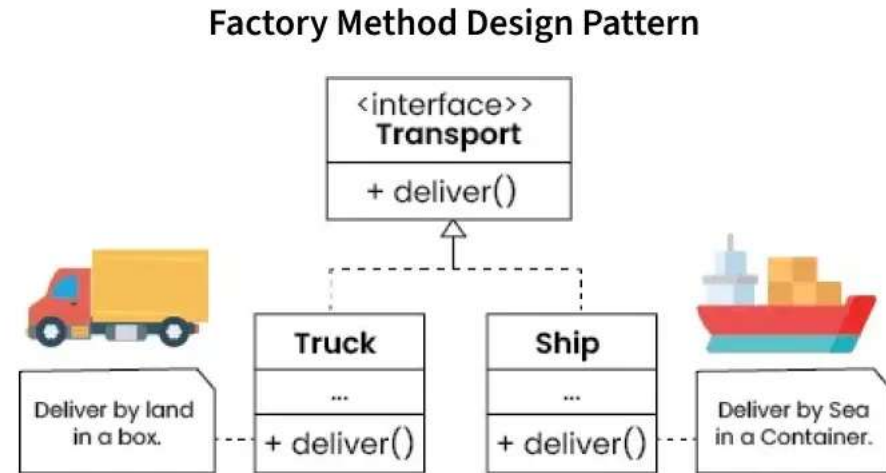
-
- Understand Factory – Method
- Understand Abstract-Factory pattern
- Compare Abstract Factory with Factory method.

Senior Lecturer: Almas Ospanov

Factory Method (1/7)

The Factory Method:

- Defines an interface for creating objects in a superclass while allowing subclasses to decide which objects to instantiate.
- Delegates the responsibility of object creation to a dedicated method, promoting loose coupling between the creator and the products, and making the system more flexible and extensible.
- Subclasses override the factory method to produce specific object types.
- Supports easy addition of new product types without modifying existing code.
- Enhances maintainability and adaptability at runtime.



Source: [geeksforgeeks.org](https://www.geeksforgeeks.org/)

Note: Instantiation refers to creating an object from a class. This involves allocating memory and initializing attributes.

Example in Java: `MvClass obi = new MvClass();` — this line instantiates obi from MvClass.

Factory Method (2/7)

When to use the Factory Method

- When it's unknown in advance the exact types and dependencies of the objects in the code.
- To provide users of the library or framework with a way to extend its internal components.
- To save system resources by reusing existing objects instead of rebuilding them every time.
- Client code could become much simpler and promote its re-usability, If your object creation process is complex or varies under different conditions.
- If an application needs to create different versions of a product and might introduce new types in the future.

Real World Software Examples

- Used in Java JDBC API for creating connections and in frameworks like Spring for managing beans.
- Libraries like Swing and JavaFX use factories to create flexible UI components.
- Tools like Log4j / SLF4J for setting up logging systems where logger factories create different logger instances based on config.
- Factories help create objects from serialized data, supporting various formats.
- An E-Commerce Platforms where need to create a ProductFactory to creates different types of products at runtime.
- Notification System may use it to create different types of notifications like WhatsApp, SMS, etc.

Factory Method (3/7)

Components of Factory Method Design Pattern

- Below are the main components of Factory Design Pattern:
- **Creator:** This is an abstract class or an interface that declares the factory method. The creator typically contains a method that serves as a factory for creating objects. It may also contain other methods that work with the created objects.
- **Concrete Creator:** Concrete Creator classes are subclasses of the Creator that implement the factory method to create specific types of objects. Each Concrete Creator is responsible for creating a particular product.
- **Product:** This is the interface or abstract class for the objects that the factory method creates. The Product defines the common interface for all objects that the factory method can create.
- **Concrete Product:** Concrete Product classes are the actual objects that the factory method creates. Each Concrete Product class implements the Product interface or extends the Product abstract class.

Implementation steps

1. Make all products follow the same interface. This interface should declare methods that make sense in every product.
2. Add an empty factory method inside the creator class. The return type of the method should match the common product interface.
3. In the creator's code find all references to product constructors. One by one, replace them with calls to the factory method, while extracting the product creation code into the factory method.
4. You might need to add a temporary parameter to the factory method to control the type of returned product.
5. At this point, the code of the factory method may look pretty ugly. It may have a large switch statement that picks which product class to instantiate. But don't worry, we'll fix it soon enough.
6. Now, create a set of creator subclasses for each type of product listed in the factory method. Override the factory method in the subclasses and extract the appropriate bits of construction code from the base method.
7. If there are too many product types and it doesn't make sense to create subclasses for all of them, you can reuse the control parameter from the base class in subclasses.

Factory Method (4/7)

Pros

- Helps to avoid tight coupling between the creator and the concrete products.
- Allows to move the product creation code into one place in the program which makes the code easier to support (Single Responsibility Principle).
- Lets to introduce new types of products into the program without breaking existing client code (Open&Close Principle).

Cons

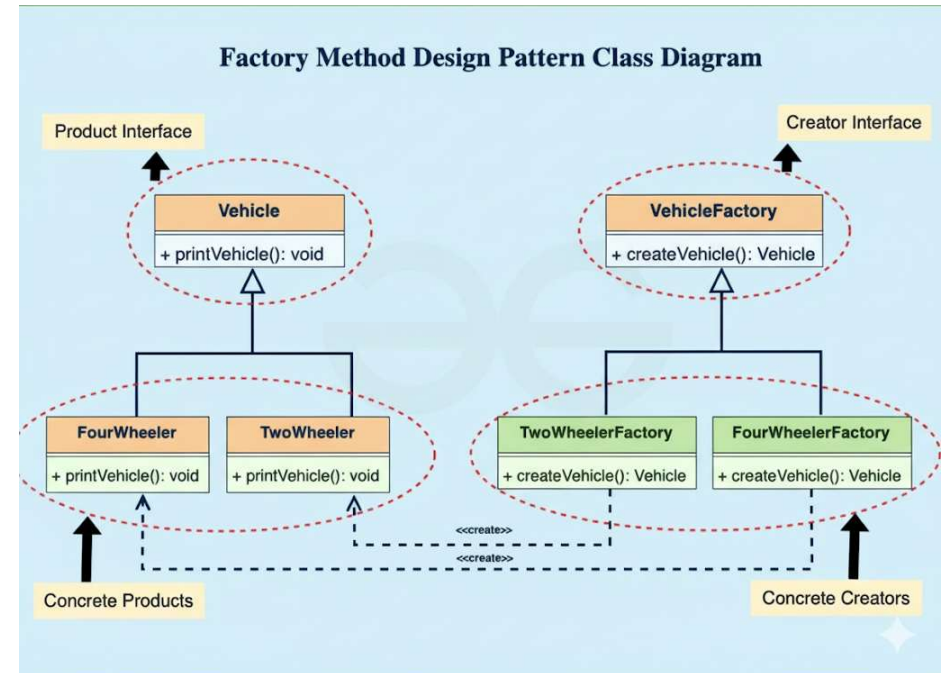
- Complexity of the code may increase, due to possibility of introducing lots of new sub-classes to implement the pattern

Factory Method (5/7)

Problem statement: Consider a software application that needs to handle the creation of various types of vehicles, such as Two Wheelers, Three Wheelers, and Four Wheelers. Each type of vehicle has its own specific properties and behaviors.

Solutions to the Problems

- Define a Factory Interface: Create an interface, **VehicleFactory**, with a method to produce vehicles.
- Create Specific Factories: Implement classes like **TwoWheelerFactory** and **FourWheelerFactory** that follow the **VehicleFactory** interface, providing methods for each vehicle type.
- Create the Client class to use a **VehicleFactory** instance instead of creating vehicles directly. This way, it can request vehicles without using conditional logic.
- **Enhance Flexibility**: This structure allows for easy addition of new vehicle types by simply creating new factory classes, without needing to alter existing Client code.



Factory Method (6/7)

1. Product interface code:

```
// Product interface representing a vehicle

public abstract class Vehicle {

    public abstract void printVehicle();}
```

2. Product interface code:

```
// Concrete product classes representing
different types of vehicles

public class TwoWheeler extends Vehicle {

    public void printVehicle() {

        System.out.println("I am two wheeler");

    }}

public class FourWheeler extends Vehicle {

    public void printVehicle() {

        System.out.println("I am four wheeler");

    }}
```

3. Creator Interface (Factory Interface) code:

```
// Factory interface defining the factory
method

public interface VehicleFactory {

    Vehicle createVehicle();}
```

4. Creator Interface (Factory Interface) code:

```
// Concrete factory class for TwoWheeler

public class TwoWheelerFactory implements
VehicleFactory {

    public Vehicle createVehicle() {

        return new TwoWheeler();  }}

// Concrete factory class for FourWheeler

public class FourWheelerFactory implements
VehicleFactory {

    public Vehicle createVehicle() {

        return new FourWheeler();  }}
```

5. Client Class

```
// Client class
class Client {

    private Vehicle pVehicle;

    public Client(VehicleFactory factory) {
        pVehicle = factory.createVehicle();  }

    public Vehicle getVehicle() {
        return pVehicle;
    }}

}
```

6. Driver

```
// Driver program
public class GFG {

    public static void main(String[] args) {

        VehicleFactory twoWheelerFactory = new
        TwoWheelerFactory();

        Client twoWheelerClient = new
        Client(twoWheelerFactory);

        Vehicle twoWheeler =
        twoWheelerClient.getVehicle();

        twoWheeler.printVehicle();

        VehicleFactory fourWheelerFactory = new
        FourWheelerFactory();

        Client fourWheelerClient = new
        Client(fourWheelerFactory);

        Vehicle fourWheeler =
        fourWheelerClient.getVehicle();

        fourWheeler.printVehicle();

    }}
```

Factory Method (7/7)

Advantages of the Factory Method

- Separates creation logic from client code, improving flexibility.
- New product types can be added easily.
- Simplifies unit testing by allowing mock product creation.
- Centralizes object creation logic across the application.
- Hides specific product classes from clients, reducing dependency.

Disadvantages of the Factory Method

- Adds more classes and interfaces, which can complicate maintenance.
- Slight performance impacts due to polymorphism.
- Concrete creators are linked to their products.
- Clients need knowledge of specific subclasses.
- May lead to unnecessary complexity if applied too broadly.
- Factory logic can be harder to test.

More Java Code Example for Factory Pattern (1/2)

// Product

```
public interface Shape { void draw(); }
```

```
public class Circle implements Shape { public void draw() { System.out.println("Circle"); } }
```

```
public class Square implements Shape { public void draw() { System.out.println("Square"); } }
```

// Factory

```
public class ShapeFactory {
```

```
    public Shape getShape(String shapeType) {
```

```
        if (shapeType.equalsIgnoreCase("CIRCLE")) return new Circle();
```

```
        if (shapeType.equalsIgnoreCase("SQUARE")) return new Square();
```

```
        return null;
```

```
    } }
```

// Usage

```
ShapeFactory factory = new ShapeFactory();
```

```
Shape shape = factory.getShape("CIRCLE");
```

```
shape.draw();
```

More Java Code Example for Factory Pattern (2/2)

// Product interface

```
interface Transport { void deliver(); }
```

```
class Truck implements Transport { public void deliver() { System.out.println("Delivering by land"); } }
```

```
class Ship implements Transport { public void deliver() { System.out.println("Delivering by sea"); } }
```

// Creator abstract class

```
abstract class Logistics {
```

```
    public abstract Transport createTransport();
```

```
    public void planDelivery() {
```

```
        Transport t = createTransport();
```

```
        t.deliver();  }}
```

// Concrete Creators

```
class RoadLogistics extends Logistics {
```

```
    public Transport createTransport() { return new Truck(); }}
```

```
class SeaLogistics extends Logistics {
```

```
    public Transport createTransport() { return new Ship(); }}
```

// Client

```
Logistics logistics = new SeaLogistics();
```

```
logistics.planDelivery();
```

// uses Ship internally

Abstract Factory Pattern (1/5)

The Abstract Factory Pattern is a creational design pattern that provides an interface for creating families of related or dependent objects without specifying their concrete classes.

Key features:

- *Acts as a "factory of factories," where a super-factory creates other factories that in turn produce specific objects.*
- *Adds a higher level of abstraction, allowing systems to switch between different product families easily while keeping the code organized and loosely coupled.*
- *Works around a super-factory that produces multiple factories.*
- *Concrete factories created at runtime decide the actual product types.*
- *Useful when the system needs to be independent of how products are created or represented.*
- *Makes it easy to switch between different groups of related objects without code changes.*

Real Life Software Examples

In applications that are supporting multiple cloud platforms where Abstract Factory Pattern facilitates the creation of cloud-specific services by provisioning resources across regions.

In applications that need to support multiple database systems (e.g., SQL, NoSQL), the Abstract Factory Pattern can be used to create database connections and queries between these databases where it could define interfaces for creating database-related objects.

In Web Browsers (Rendering Engines) where it generates browser-specific rendering components. Use cases: DOM, CSS, and JS object creation.

Java XML Parsers where the pattern creates parser families without exposing concrete classes.

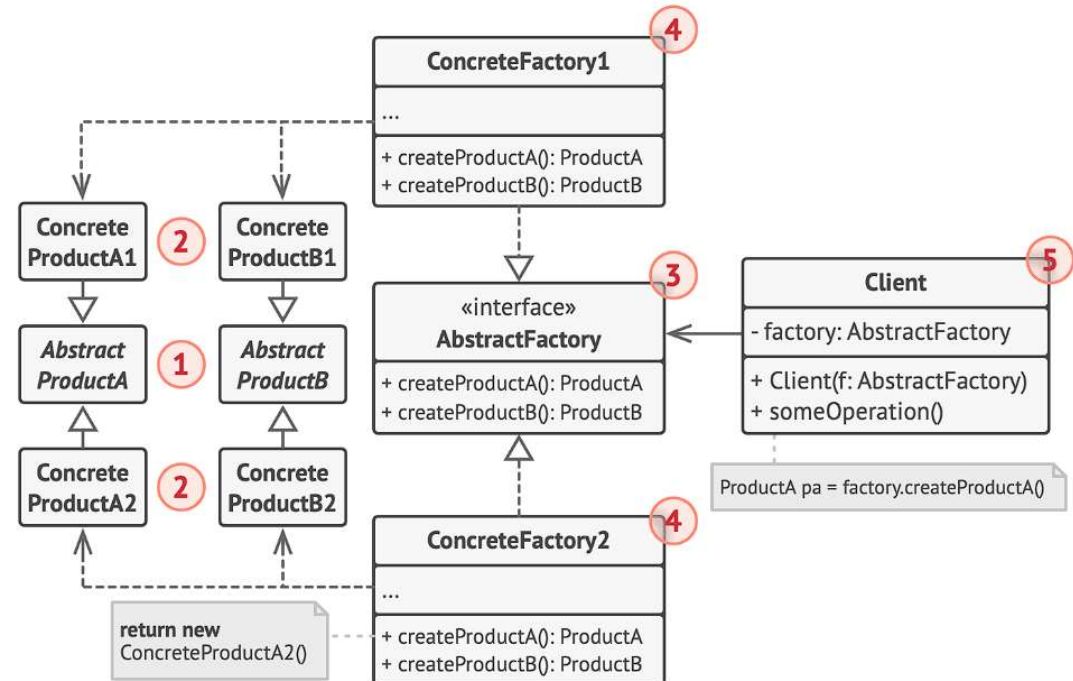
GUI Toolkits in Cross-platform UI rendering where pattern creates platform-specific Button, Checkbox, etc. .

Abstract Factory Pattern (2/5)

Abstract Factory Pattern's Components

- Abstract Factory: Supplies a mode in such that concrete factories follow a common interface, providing consistent way to produce related set of objects.
- Concrete Factories: Implement the rules specified by the abstract factory. It contain the logic for creating specific instances of objects within a family.
- Abstract Products: Acts as an abstract or interface type that all concrete products within a family must follow to and provides a unified way for concrete products to be used interchangeably.
- Concrete Products: Implement the methods declared in the abstract products, ensuring consistency within a family and belong to a specific category or family of related objects.
- Client: Utilizes the abstract factory to create families of objects without specifying their concrete types and interacts with objects through abstract interfaces provided by abstract products.

Structure



Source: refactoring.guru

Abstract Factory Pattern (3/5)

Challenges of using Abstract Factory Pattern

Below are the main challenges of using abstract factory pattern:

The Abstract Factory pattern can add unnecessary complexity to simpler projects with multiple factories and interfaces.

Adding new product types may require changes to both concrete factories and the abstract factory interface, impacting existing code.

Introducing more factories and product families can quickly increase the number of classes, making code management difficult in smaller projects.

It may violate the Dependency Inversion Principle if client code depends directly on concrete factories rather than abstract interfaces.

Benefits of using Abstract Factory Pattern

- Separates the creation of objects, so clients don't need to know specific classes.
- Clients interact with objects through abstract interfaces, keeping class names hidden from client code.
- Changing the factory allows for different product configurations, as all related products change together.
- Ensures that an application uses objects from only one family at a time for better compatibility.

Abstract Factory Pattern (4/5)

When not to use Abstract Factory Pattern

- The product families are unlikely to change, as it may add unnecessary complexity.
- When application only requires single, independent objects and isn't concerned with families of related products.
- When overhead of maintaining multiple factories outweighs the benefits, particularly in smaller applications.
- When simpler solutions, like the Factory Method or Builder pattern, if they meet your needs without adding the complexity of the Abstract Factory pattern.

When to use Abstract Factory Pattern

- When a system requires multiple families of related products and you want to ensure compatibility between them.
- When there's a need for flexibility and extensibility, allowing for new product variants to be added without changing existing client code.
- When it's to encapsulate the creation logic, making it easier to modify or extend the object creation process without affecting the client.
- When to maintain consistency across different product families, ensuring a uniform interface for the products.

Abstract Factory Pattern (5/5)

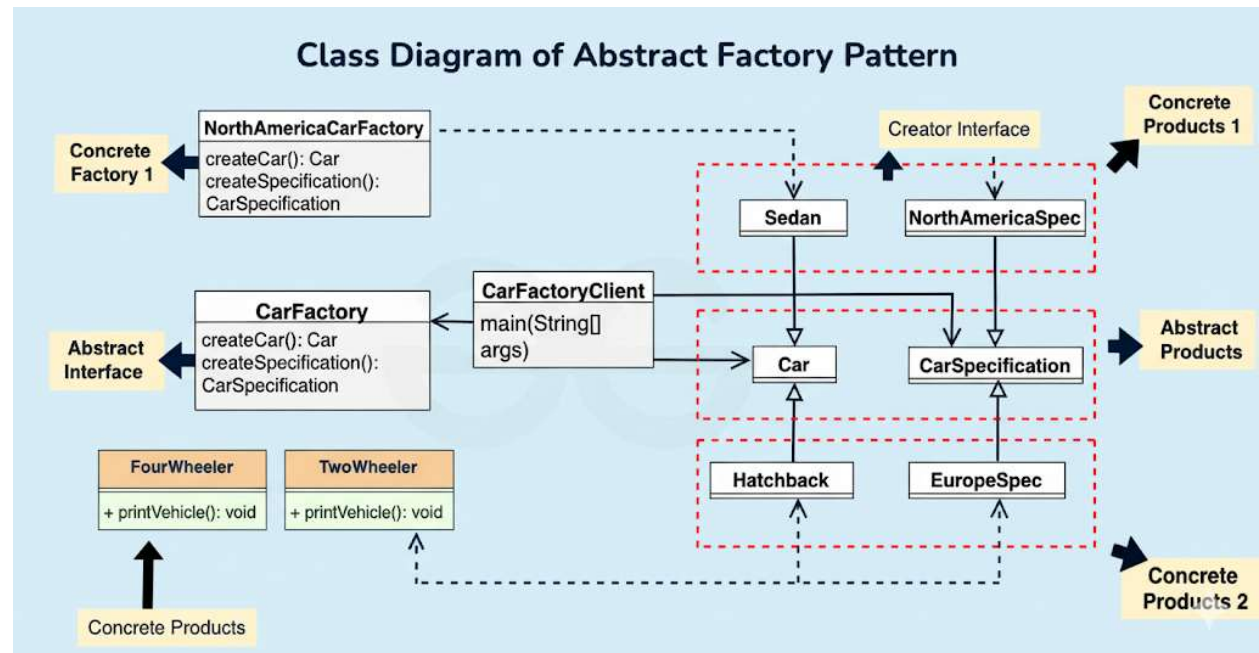
Problem statement:

There's a need design a system to create cars with specific configurations for different regions, such as North America and Europe.

Each region may have unique requirements and regulations, and need to ensure that cars produced for each region meet those standards.

Solution with Abstract Factory Pattern:

- Different regions has their own factory to create cars for local needs.
- Therefore, it keeps the design and features the same for vehicles in each region.
- It's possible to change one region without affecting others (e.g., updating North America doesn't impact Europe).
- To add a new region, just need to create a new factory, no need to change existing code.
- The pattern keeps car creation separate from how they are used.



Code example for Abstract Factory Structure (1/2)

// Abstract Products

```
interface Chair { void sit(); }  
interface Sofa { void lieOn(); }
```

// Concrete Products

```
class VictorianChair implements Chair { public void sit()  
{ System.out.println("Victorian chair"); } }  
class ModernChair implements Chair { public void sit()  
{ System.out.println("Modern chair"); } }  
class VictorianSofa implements Sofa { public void lieOn()  
{ System.out.println("Victorian sofa"); } }  
class ModernSofa implements Sofa { public void lieOn()  
{ System.out.println("Modern sofa"); } }
```

// Abstract Factory

```
interface FurnitureFactory {  
    Chair createChair();  
    Sofa createSofa();  
}
```

// Concrete Factories

```
class VictorianFurnitureFactory implements FurnitureFactory  
{  
    public Chair createChair() { return new VictorianChair(); }  
    public Sofa createSofa() { return new VictorianSofa(); }  
}  
class ModernFurnitureFactory implements FurnitureFactory  
{  
    public Chair createChair() { return new ModernChair(); }  
    public Sofa createSofa() { return new ModernSofa(); }  
}
```

// Client

```
FurnitureFactory factory = new ModernFurnitureFactory();  
  
Chair chair = factory.createChair();  
  
Sofa sofa = factory.createSofa();  
  
chair.sit(); // prints "Modern chair"
```

Code example for Abstract Factory Structure (2/2)

```
// Abstract Products
public interface Button { void paint(); }
public interface Checkbox { void paint(); }

// Concrete Products for Windows
public class WinButton implements Button { public void
paint() { System.out.println("Win Button"); } }
public class WinCheckbox implements Checkbox { public
void paint() { System.out.println("Win Checkbox"); } }

// Abstract Factory
public interface GUIFactory { Button createButton();
Checkbox createCheckbox(); }
```

```
// Concrete Factory

public class WinFactory implements GUIFactory {

    public Button createButton() { return new
WinButton(); }

    public Checkbox createCheckbox() { return new
WinCheckbox(); }}

// Usage

GUIFactory factory = new WinFactory();

Button button = factory.createButton();

button.paint();
```

Summary (1/2)

Factory Method can be used with other patterns such as:

- Some Software Design approaches start by using Factory method for its less complication and start developing code towards Abstract Factory, Prototype or Builder patterns.
- Along with Iterator pattern to let collection subclasses to return different types of iterators.
- Factory Method is a specialization of Template Method. So, it can serve as a step in a large Template Method

Abstract Factory can be used with other patterns such as:

- Builder pattern that constructs complex objects step by step whereas Abstract Factory focuses in creating families of related objects.
- Abstract Factory classes are usually will be based on a set of Factory Methods, but, yet Prototype pattern can be used to compose the methods on these classes.
- Bridge pattern that can work only with specific implementation whereas Abstract Factory can encapsulate these relations and hide the complexity from the client code.

Summary (2/2)

Aspect	Factory Method	Abstract Factory
Purpose	Create one product by delegating instantiation to subclasses	Create families of related products without specifying concrete classes
Design Principle	Relies on inheritance	Relies on composition
Level of Abstraction	Lower—focuses on a single product	Higher—manages multiple related products
Flexibility	Easy to extend with new product types	Easy to switch entire product families
Client Knowledge	Client may need to know which subclass to instantiate	Client uses a factory interface—no need to know concrete classes
Example Use Case	Logger creation (getLogger() returns one logger)	UI toolkit (WidgetFactory creates Button, Checkbox, etc.)

Real-World Analogy:

- Factory Method: Like a single bakery method that makes one type of pastry depending on the subclass (e.g., CroissantFactory, MuffinFactory).
- Abstract Factory: Like a bakery chain that provides entire breakfast sets—each set includes a pastry, a drink, and a side, all coordinated.
- If you're designing an LMS module, for example:
 - Use Factory Method to instantiate different types of content (e.g., Quiz, Video, Assignment).
 - Use Abstract Factory to generate entire course themes (e.g., STEMFactory creates STEM-style Quiz, Video, Assignment objects).

Scenario Case	Recommended Pattern
Need to create one type of object	Factory Method
Need to create related objects as a group	Abstract Factory
Want to extend product types independently	Factory Method
Want to switch product families easily	Abstract Factory

Clean code ,Chapter 6 (1/2)

Objects vs Data Structures — Design Philosophy

Data Abstraction

- Good design hides implementation details behind abstract interfaces.
- Avoid exposing internal variables via getters/setters—this breaks encapsulation.
- Example:
 - `Point { double x, double y; }` → exposes implementation
 - `Point { getX(), getY(), setCartesian(), setPolar() }` → hides it

Law of Demeter (Talk to Friends, Not Strangers)

- A method should only call:
 - Its own methods
 - Methods of objects it creates, receives, or owns
- Avoid “train wrecks” like:
 - `ctxt.getOptions().getScratchDir().getAbsolutePath()`
 - → Instead: `ctxt.createScratchFileStream(classFileName)`

Data/Object Anti-Symmetry

- Objects: expose behavior, hide data
- Data Structures: expose data, have no behavior
- Implication:
 - OO code → easy to add new types, hard to add new behaviors
 - Procedural code → easy to add new behaviors, hard to add new types
- Choose based on system needs—mature design balances both

Clean code ,Chapter 6 (2/2)

Practical Guidelines for Clean Data Design

Avoid Hybrids

- Don't mix object behavior with exposed data (via public fields or accessors)
- Hybrids make it hard to extend either behavior or structure—worst of both worlds

Data Transfer Objects (DTOs)

- Pure data containers: public fields, no behavior
- Useful for:
 - Database communication
 - Message parsing
 - Serialization

Active Record Pattern

- DTOs with navigation methods (e.g., save(), find())
- Keep business logic separate—don't mix domain rules into data containers

Conclusion

- Understand when to use objects vs data structures
- Encapsulation isn't just about access modifiers—it's about abstraction
- Clean code means choosing the right paradigm for the right part of the system